

PROYECTO MINI SHELL EN C++

SISTEMAS OPERATIVOS

Presentado por:

Vladimir Roger Ticona Mamani

Edu Rubinho Puma Ccama

Jorge Enrique Obando Huallpa





INTRODUCCIÓN

En los sistemas operativos basados en Unix y Linux, el intérprete de comandos o shell constituye una de las herramientas fundamentales para la interacción entre el usuario y el sistema. A través del shell, los usuarios pueden ejecutar programas, administrar archivos, gestionar procesos y realizar tareas complejas mediante comandos simples o scripts. Comprender su funcionamiento interno permite profundizar en conceptos esenciales de la programación de sistemas, como la creación y sincronización de procesos, el manejo de hilos, la gestión de memoria, y el control de entrada/salida (I/O).

JUSTIFICACION LINUX

El desarrollo del mini-shell se realiza en Linux debido a que este sistema operativo ofrece un entorno nativo y completo para la programación a nivel de sistema, especialmente en lo referente al manejo de procesos, hilos y llamadas al sistema.

Linux proporciona una interfaz POSIX (Portable Operating System Interface) estandarizada, que incluye las funciones esenciales como `fork()`, `exec()`, `wait()`, `pipe()`, `dup2()`, y las bibliotecas `pthread` para la concurrencia. Estas herramientas son fundamentales para implementar el funcionamiento interno de un intérprete de comandos.



OBJETIVOS

Desarrollar un intérprete de comandos (mini-shell) en C++ para sistemas Linux, implementando el manejo de procesos, hilos, concurrencia y gestión de memoria, como se establece en la Unidad I del curso.

- Crear un shell que ejecute comandos mediante `fork()` y `exec*()`, manejando la sincronización con `wait()` y `waitpid()`.
- Implementar redirección de salida estándar utilizando `dup2`, `pipe` y `close`.
- Utilizar mecanismos de concurrencia como hilos (`pthread`) para la ejecución en paralelo de comandos.
- Manejar errores de ejecución y redirección de salida, mostrando mensajes claros.

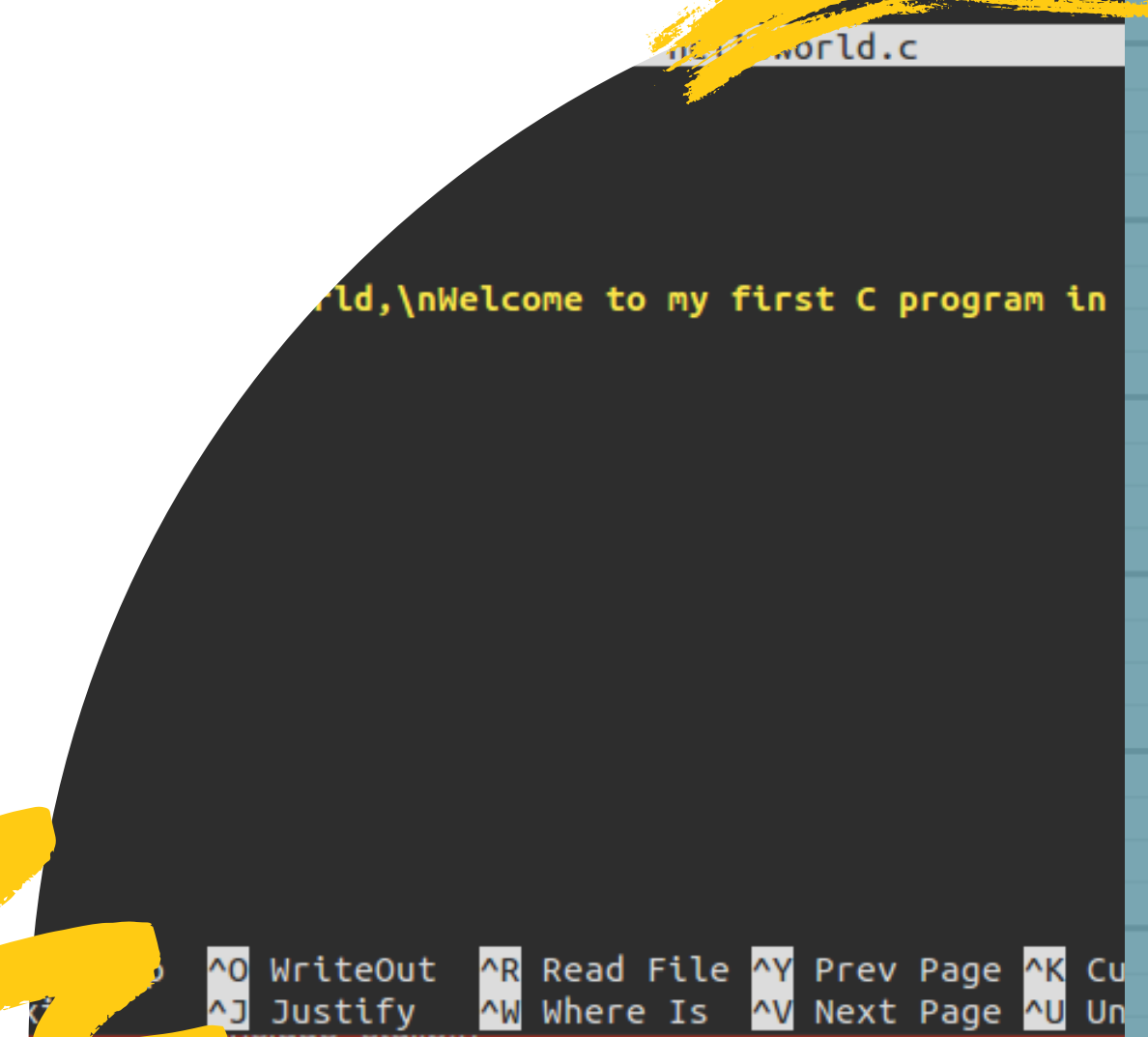
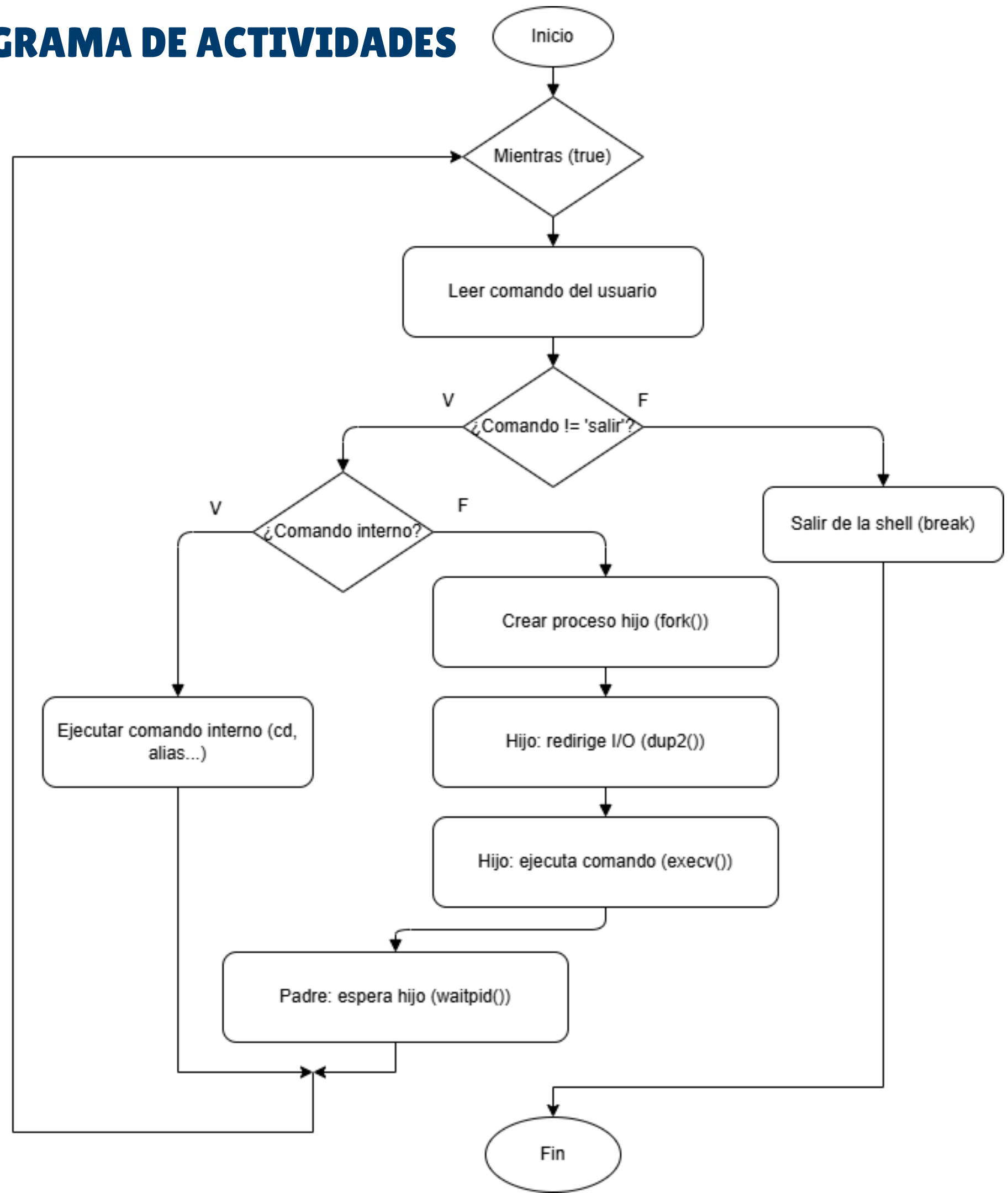
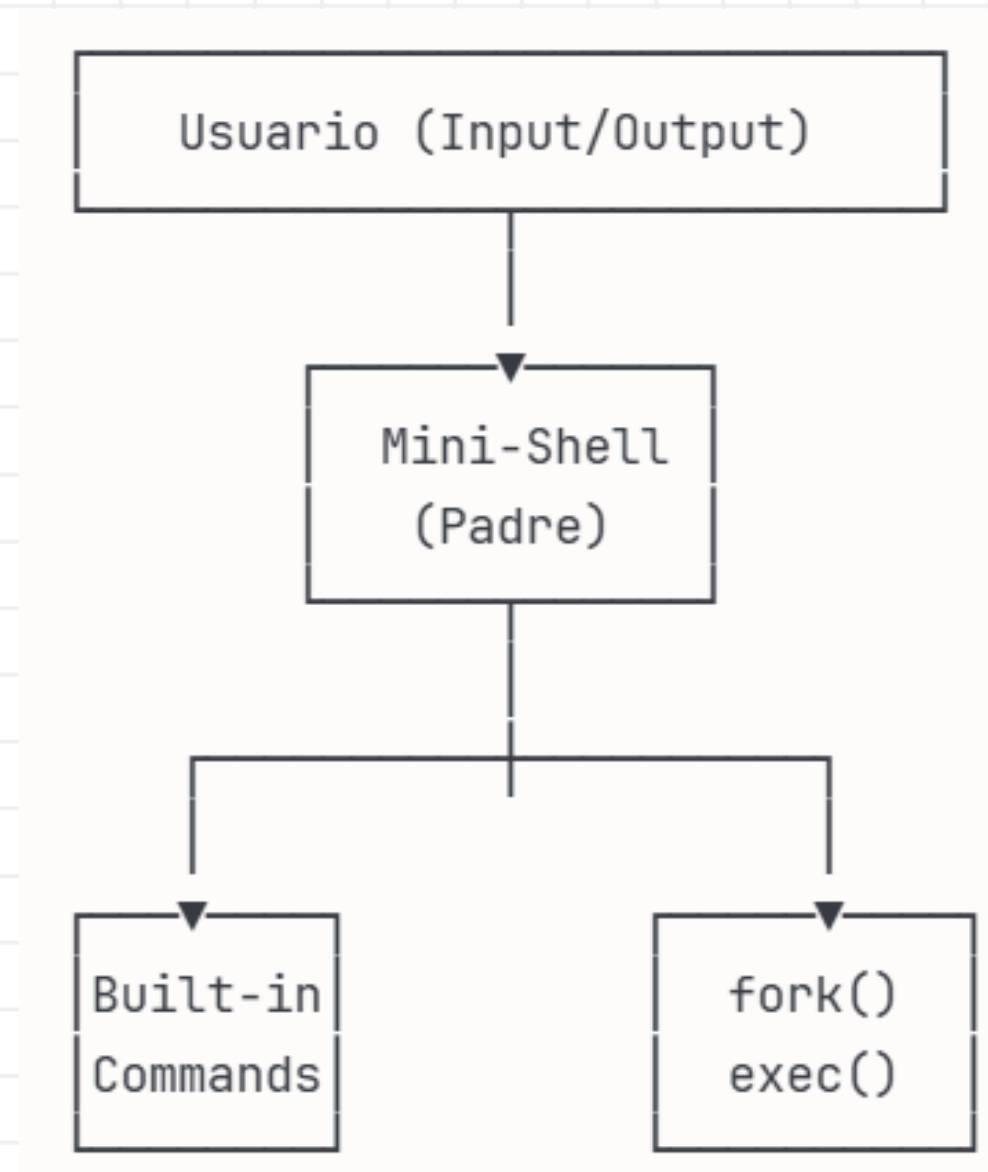


DIAGRAMA DE ACTIVIDADES



- `fork()`: Crea un nuevo proceso hijo. Es la base para la ejecución de comandos externos.
- `dup2()`: Se utiliza para la redirección de salida (>). Esta llamada duplica un descriptor de archivo, lo que permite redirigir la salida estándar del proceso hijo (`STDOUT_FILENO`) al descriptor del archivo de destino, previamente abierto.
- `execv()`: Reemplaza el proceso actual con un nuevo programa. Se utiliza en el proceso hijo para ejecutar el comando del usuario.
- `waitpid()`: Permite que el proceso padre espere la finalización del proceso hijo, evitando procesos huérfanos y garantizando el comportamiento por defecto en primer plano.

● ARQUITECTURA GENERAL



El proceso padre gestiona el loop principal y ejecuta built-ins directamente. Para comandos externos, crea procesos hijo con `fork/exec`

● FUNCIONALIDADES BASE

1. Prompt personalizado mini-shell [/home/usuario]>

2. Resolución de rutas

✓ Ruta absoluta: /usr/bin/whoami

✓ Búsqueda en /bin: ls

3. Ejecución con fork() + execv()

4. Redirección de salida (>) ls > archivo.txt

5. Manejo de errores en español

● FUNCIONALIDADES PRINCIPALES

1. Funciones Principales:

- `mostrar_prompt()`: Muestra el prompt personalizado con el directorio actual.
- `tokenizar()`: Separa la línea de entrada en tokens utilizando espacios y tabulaciones.
- `builtin_cd()`: Permite cambiar el directorio de trabajo.
- `builtin_pwd()`: Muestra el directorio de trabajo actual.
- `builtin_help()`: Muestra los comandos disponibles y su descripción.
- `builtin_history()`: Muestra el historial de comandos ejecutados.
- `builtin_meminfo()`: Muestra estadísticas de memoria del proceso.
- `builtin_alias()` y `builtin_unalias()`: Gestionan alias de comandos.

● COMANDOS INTERNOS

cd	Cambiar directorio
pwd	Mostrar directorio actual
help	Ayuda del sistema
history	Historial de comandos
alias	Crear atajos de comandos

● PRUEBAS CRITICAS

- ✓ Comando inexistente Error: comando 'noexiste' no encontrado
- ✓ Sin permisos de escritura Error: no se pudo abrir el archivo
- ✓ Redirección exitosa `ls > archivo.txt` (nada en pantalla)
- ✓ Alias con expansión `alias l='ls' → l -la funciona`

● LLAMADAS DEL SISTEMA UTILIZADAS

Procesos:

- `fork()` - Crear proceso hijo
- `execv()` - Ejecutar programa
- `waitpid()` - Esperar terminación

Descriptores:

- `open()` - Abrir archivos
- `close()` - Cerrar descriptores
- `dup2()` - Redirigir E/S

Sistema de archivos:

- `chdir()` - Cambiar directorio
- `getcwd()` - Obtener directorio actual
- `stat()` - Verificar permisos

● CAPTURAS DE EJECUCION DE ALGUNOS COMANDOS

```
(ticma@kali) - [~/Escritorio]
$ ./minishell
=====
Bienvenido a Mini-Shell
Escribe 'help' para ver la ayuda
Escribe 'salir' para terminar
=====

mini-shell [/home/ticma/Escritorio]> whoami
ticma
mini-shell [/home/ticma/Escritorio]> hostname
kali
mini-shell [/home/ticma/Escritorio]> echo hola mundo
hola mundo
mini-shell [/home/ticma/Escritorio]> cd ..
mini-shell [/home/ticma]> ls
2025-09-06-194116_1920x1080_scrot.png  guardado.txt          Público
abc                                     Imágenes              resultado.txt
Descargas                             Música                 resultado.xml
Documentos                            packages.microsoft.gpg Vídeos
Ejemplo                               Plantillas
Escritorio                            practica
mini-shell [/home/ticma]> cd Escritorio
mini-shell [/home/ticma/Escritorio]> comandofalso
Error: comando 'comandofalso' no encontrado
```

● CAPTURAS DE EJECUCION DE ALGUNOS COMANDOS

```
mini-shell [/home/ticma/Escritorio]> ls > prueba.txt
mini-shell [/home/ticma/Escritorio]> cat prueba.txt
03-concurrencia-vladimirticona-main
03-concurrencia-vladimirticona-main.zip
archivo.txt
kali-hydra.desktop
kali-nikto.desktop
kali-nmap.desktop
listado_completo.txt
Mensajes
minishell
minishell.cpp
prueba.txt
salida.txt
ZZ
mini-shell [/home/ticma/Escritorio]> echo texto de prueba > archivo.txt
mini-shell [/home/ticma/Escritorio]> cat archivo.txt
texto de prueba
mini-shell [/home/ticma/Escritorio]> pwd
/home/ticma/Escritorio
```


● CAPTURAS DE EJECUCION DE ALGUNOS COMANDOS

```
mini-shell [/home/ticma/Escritorio]> ayuda

=== MINI-SHELL - AYUDA ===

Comandos internos:
  cd                - Cambiar directorio
  pwd               - Mostrar directorio actual
  help              - Mostrar ayuda
  history           - Mostrar historial de comandos
  meminfo           - Mostrar estadísticas de memoria del proceso
  alias [nombre='comando'] - Crear alias
  unalias [nombre]  - Eliminar un alias
  salir             - Salir de la shell

Características:
  comando > archivo - Redirigir salida estándar a archivo
  /ruta/absoluta/cmd - Ejecutar comando con ruta absoluta
  comando            - Buscar comando en /bin/

Ejemplos:
  ls > listado.txt
  alias ll='ls'
  /usr/bin/whoami
  cat /etc/hostname

=====
```

● CAPTURAS DE EJECUCION DE ALGUNOS COMANDOS

```
mini-shell [/home/ticma/Escritorio]> history
```

```
=== HISTORIAL DE COMANDOS ===
```

```
1  whoami
2  hostname
3  echo hola mundo
4  cd ..
5  ls
6  cd Escritorio
7  comandofalso
8  ls > prueba.txt
9  cat prueba.txt
10 echo texto de prueba > archivo.txt
11 cat archivo.txt
12 pwd
13 ayuda
14 history
```

```
=====
```


● CAPTURAS DE EJECUCION DE ALGUNOS COMANDOS

```
mini-shell [/home/ticma/Escritorio]> meminfo
```

```
=== ESTADÍSTICAS DE MEMORIA DEL PROCESO ===
```

```
VmPeak:      6276 kB  
VmSize:      6276 kB  
VmRSS:       3304 kB  
VmData:      268 kB  
VmStk:       132 kB  
VmExe:       32 kB
```

```
Descripción:
```

```
VmPeak: Pico máximo de memoria virtual usada  
VmSize: Tamaño actual de memoria virtual  
VmRSS:  Memoria física realmente en uso  
VmData: Memoria del heap (datos dinámicos)  
VmStk:  Memoria del stack  
VmExe:  Memoria del código ejecutable
```

```
=====
```

```
mini-shell [/home/ticma/Escritorio]> salir  
Saliendo.....
```



DEMOSTRACION EN VIVO DE LA EJECUCION DE LA MINISHELL





CONCLUSIONES

El mini-shell cumple con los objetivos establecidos, implementando correctamente las funcionalidades básicas y avanzadas, incluyendo redirección de salida, concurrencia, y gestión de memoria. Se logró una correcta sincronización de hilos y una gestión eficiente de recursos. Se podría extender la funcionalidad con más comandos internos, como kill para terminar procesos, o mejoras en la gestión de memoria, implementando contadores de referencias o utilizando técnicas más avanzadas como mmap para gestionar grandes cantidades de memoria de manera eficiente.

Gracias